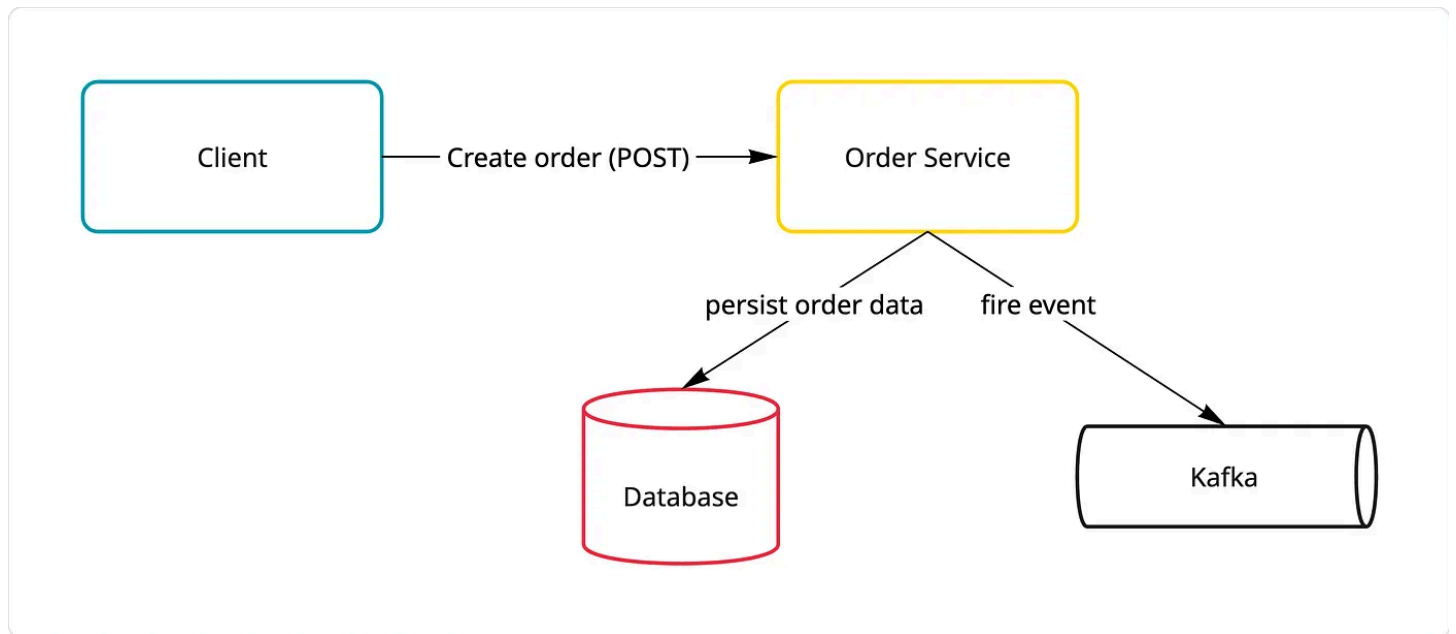


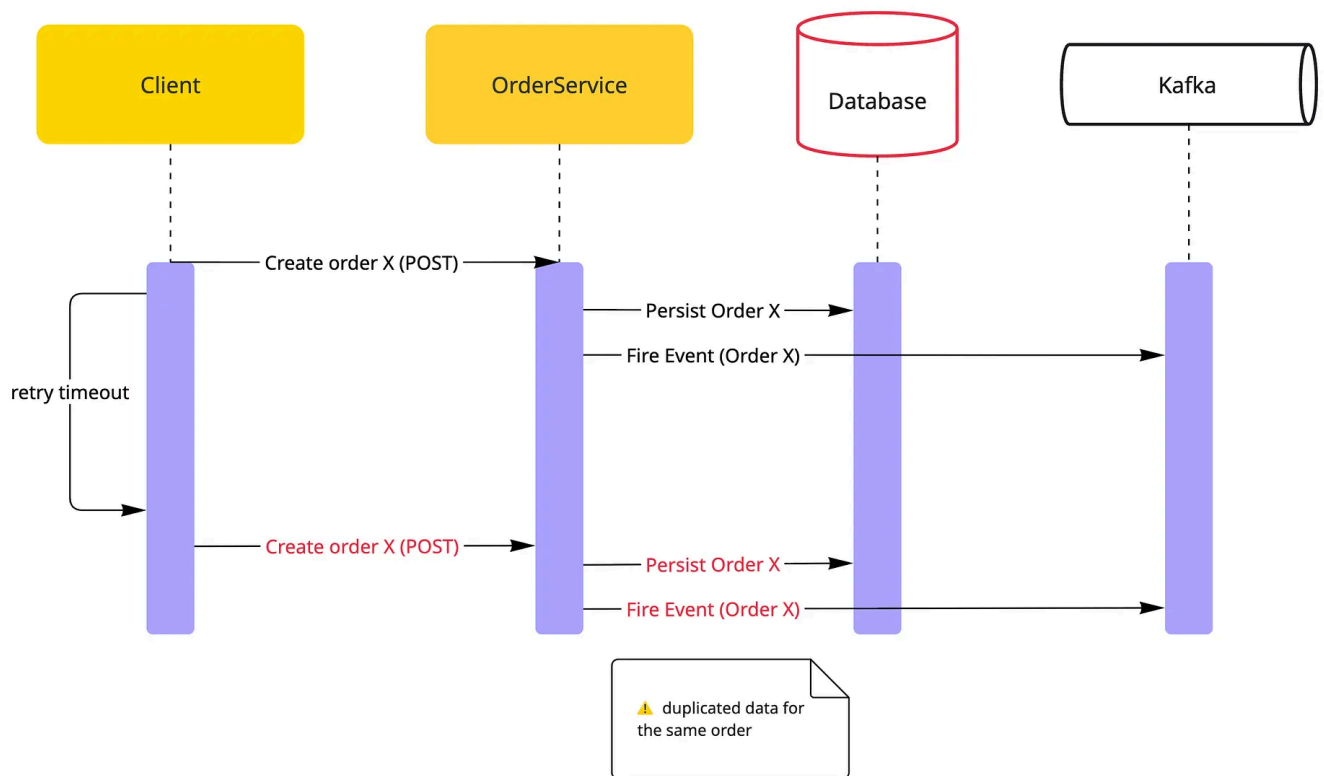
Implementing Idempotent POST Requests

- Idempotence = operation that can be repeated with the same effect
- GET/PUT/DELETE → idempotent
- POST → *not* idempotent
- Duplicate POSTs → duplicate orders / events



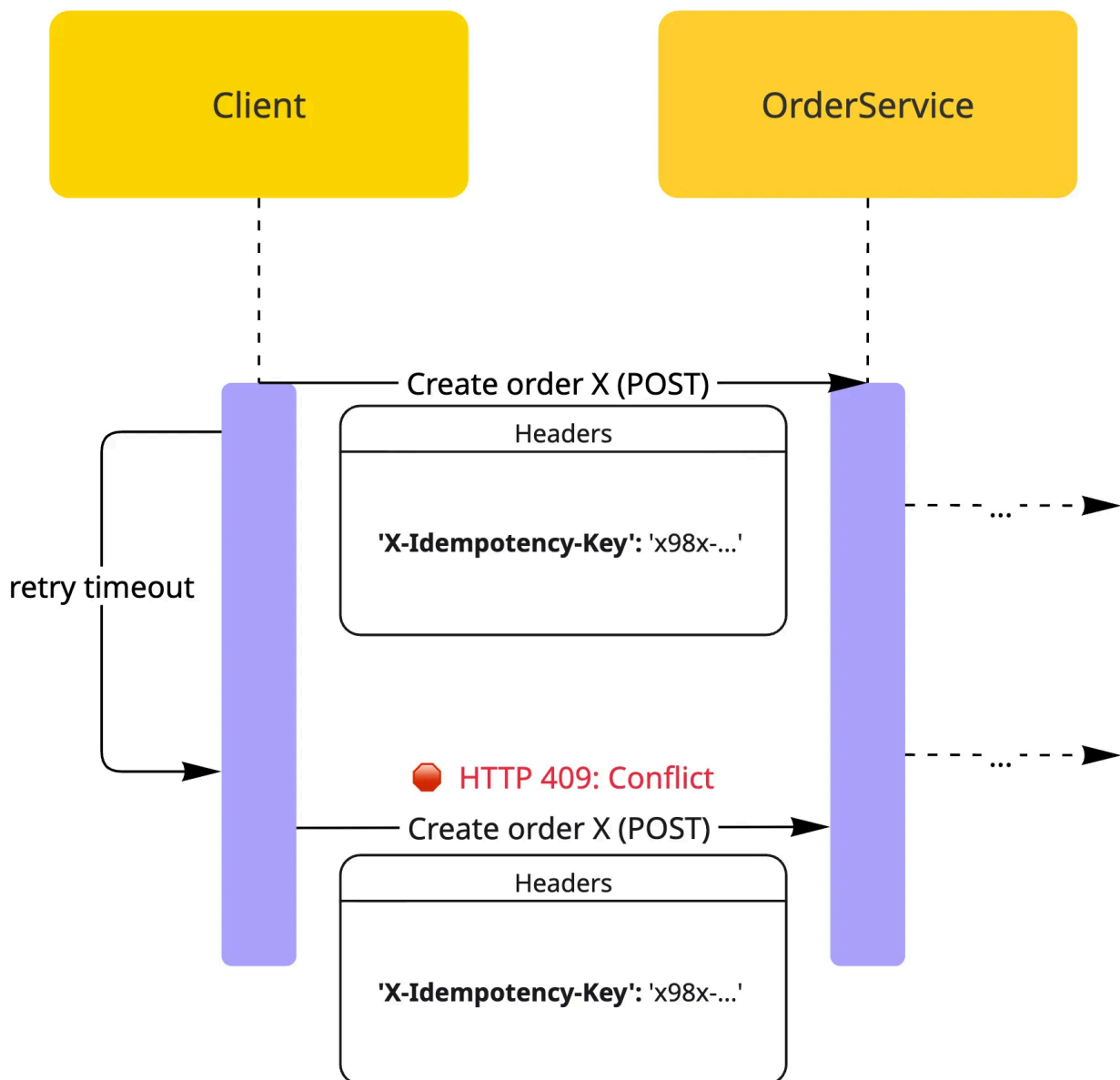
The Problem

- Retries happen because of:
 - client timeouts
 - network failures
 - server crashes
- POST /orders sent twice may create two orders
- DB constraints don't help when the same order is valid twice
- Downstream (Kafka, billing, logistics) → becomes a mess



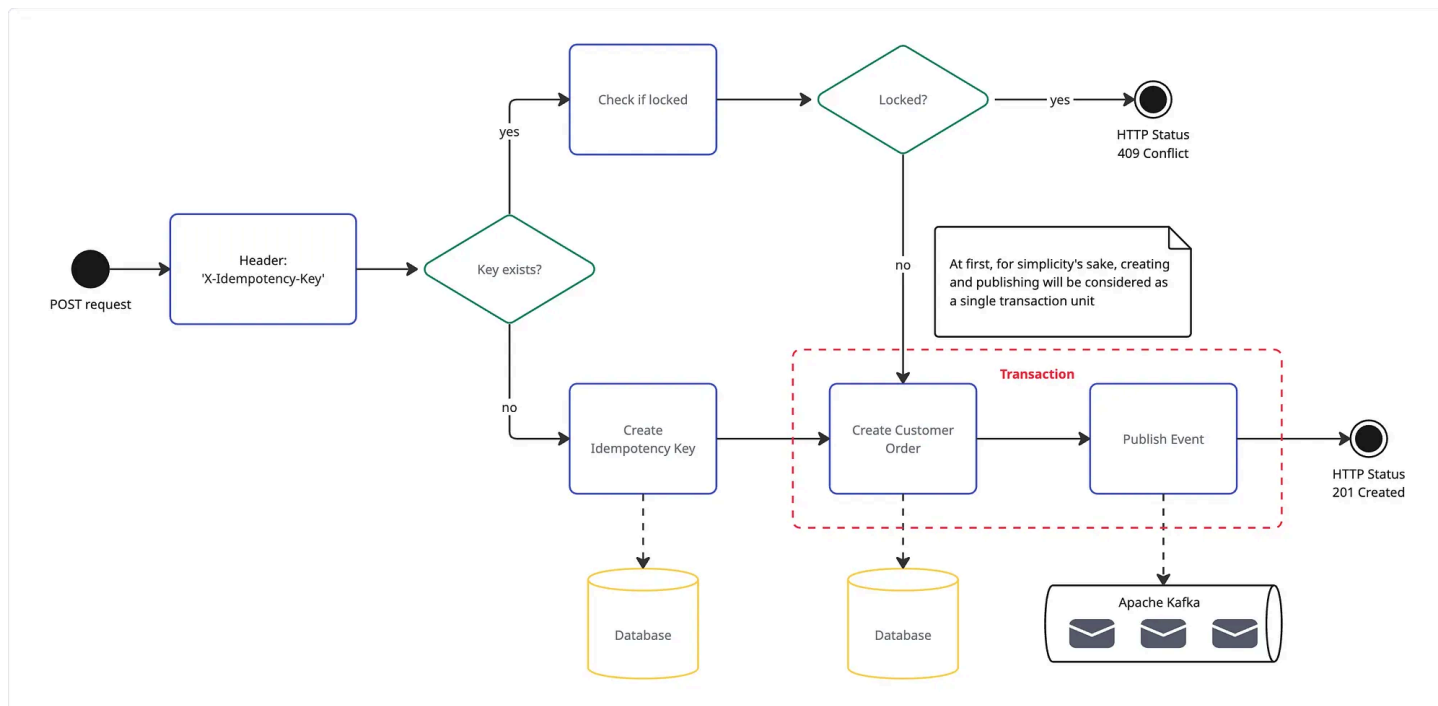
Solution Overview

- Client sends a **unique idempotency key**
 - Header: `X-Idempotency-Key: <uuid>`
- Server stores the key
- If key already present → immediately return **409 Conflict**
- Guarantee: each *intent* is processed once



Architecture

- Client → sends key + request body
- Server:
 1. try to create key record (transaction)
 2. if locked → 409
 3. else create order + publish event
 4. return 201



Controller Example

```

@PostMapping
public ResponseEntity<?> createOrder(
    @RequestHeader("X-Idempotency-Key") String key,
    @RequestBody CreateCustomerOrderDto order) {

    IdempotencyKey entity;
    try {
        entity = idempotencyKeyService
            .getOrCreateIdempotencyKeyByKeyAndUserId(key, order.getUserId());
    } catch (IllegalStateException e) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("status", "duplicate"));
    }

    Long orderId = ordersService.createOrder(order);
    kafkaProducerService.sendMessage("orders-topic", orderId);

    return ResponseEntity.status(HttpStatus.CREATED)
        .body(Map.of("orderId", orderId));
}

```

Idempotency Service

```

@Transactional
public IdempotencyKey getOrCreateIdempotencyKeyByKeyAndUserId(
    String key, String userId) {

    Optional<IdempotencyKey> existing =
        repo.findByIdempotencyKeyAndUserId(key, userId);

    IdempotencyKey idKey;
}

```

```
if (existing.isPresent()) {  
    idKey = existing.get();  
    if (checkIfIsLocked(idKey)) {  
        throw new IllegalStateException("Key is locked");  
    }  
} else {  
    idKey = new IdempotencyKey(key, userId);  
}  
  
idKey.setLockedAt(LocalDateTime.now());  
return repo.save(idKey);  
}
```

Resources